

AIU Global Hub Backend Report

1. Project Overview

The AIU Global Hub backend is a server-side application developed to support the AIU Global Hub web platform. The platform is designed to help students access international opportunities such as internships, exchange programs, dual-degree programs, and research opportunities.

The backend provides secure authentication, opportunity management, student applications, and database operations using a lightweight and scalable architecture.

The system was implemented using:

- Node.js
- Express.js
- SQLite
- bcrypt
- CORS

The backend communicates with the frontend using REST API endpoints and JSON responses.

2. Objectives of the Backend System

The main objectives of the backend are:

1. Manage user registration and login.
2. Store and retrieve opportunities.
3. Allow students to apply for opportunities.
4. Allow administrators and managers to create and manage opportunities.
5. Secure passwords using hashing.
6. Provide structured API endpoints for frontend integration.
7. Maintain persistent data storage using SQLite.

3. Backend Architecture

The backend follows a simple layered architecture.

3.1 Architecture Components

Client Layer

The frontend web application sends HTTP requests to the backend using `fetch()` API.

Server Layer

The Express.js server handles:

- Routing
- Request validation
- Authentication
- Business logic
- Database communication

Database Layer

SQLite is used as the relational database for storing:

- Users
- Opportunities
- Applications

4. Technologies Used

Technology	Purpose
Node.js	Runtime environment
Express.js	Backend framework
SQLite	Relational database
bcrypt	Password hashing
CORS	Cross-origin communication
Nodemon	Development server auto-restart

5. Project Structure

```
backend/
|
├── server.js
├── db.js
├── initDb.js
```

```
|— database.sqlite
|— package.json
|— node_modules/
```

5.1 server.js

Main backend server file.

Responsibilities:

- Creates Express application
- Defines API routes
- Handles authentication
- Handles CRUD operations
- Sends JSON responses
- Connects frontend with database

5.2 db.js

Database helper file.

Responsibilities:

- Connects SQLite database
- Executes SQL queries
- Provides reusable Promise-based helper functions

Functions:

- run()
- get()
- all()

5.3 initDb.js

Database initialization file.

Responsibilities:

- Creates database tables
- Seeds default users
- Seeds sample opportunities

5.4 database.sqlite

Main database file containing all stored records.

6. Database Design

The backend database contains three main tables.

6.1 Users Table

Purpose:

Stores user account information.

Field	Type	Description
id	INTEGER	Primary key
name	TEXT	User full name
email	TEXT	User email
password_hash	TEXT	Encrypted password
role	TEXT	User role
university_id	TEXT	University ID
created_at	TEXT	Account creation timestamp

User Roles

- student
- admin
- manager

6.2 Opportunities Table

Purpose:

Stores available opportunities.

Field	Type	Description
id	INTEGER	Primary key
title	TEXT	Opportunity title
category	TEXT	Opportunity category

Field	Type	Description
field	TEXT	Academic field
location	TEXT	Opportunity location
provider	TEXT	Provider institution
term	TEXT	Duration or term
description	TEXT	Opportunity details
requirements	TEXT	Eligibility requirements
status	TEXT	Active/inactive status
created_at	TEXT	Creation timestamp

6.3 Applications Table

Purpose:

Stores student applications.

Field	Type	Description
id	INTEGER	Primary key
user_id	INTEGER	Student ID
opportunity_id	INTEGER	Opportunity ID
statement	TEXT	Student statement
recommendation_professor	TEXT	Professor recommendation
graduation_year	TEXT	Expected graduation year
status	TEXT	Application status
created_at	TEXT	Submission timestamp

Foreign Key Relationships

- user_id → users.id
- opportunity_id → opportunities.id

7. Authentication System

The backend implements a basic authentication system.

7.1 Registration Process

Endpoint:

```
POST /api/auth/register
```

Process:

1. Validate input fields.
2. Check AIU email domain.
3. Check if email already exists.
4. Hash password using bcrypt.
5. Store user in database.
6. Return success response.

Password Security

Passwords are never stored as plain text.

The system uses:

```
bcrypt.hash(password, 10)
```

This protects user credentials from exposure.

7.2 Login Process

Endpoint:

```
POST /api/auth/login
```

Process:

1. Receive email and password.
2. Search for matching user.
3. Compare password using `bcrypt.compare()`.
4. Return authenticated user information.

8. Opportunity Management System

The backend supports full CRUD operations.

CRUD means:

- Create
- Read
- Update
- Delete

8.1 Retrieve Opportunities

Get All Opportunities

`GET /api/opportunities`

Returns all available opportunities.

Get Single Opportunity

`GET /api/opportunities/:id`

Returns detailed information about one opportunity.

8.2 Create Opportunity

`POST /api/opportunities`

Only managers or admins can create opportunities.

The backend validates:

- title
- category
- field
- location
- provider
- term

8.3 Update Opportunity

`PUT /api/opportunities/:id`

Allows editing existing opportunity information.

8.4 Delete Opportunity

`DELETE /api/opportunities/:id`

Allows managers/admins to remove opportunities.

9. Application Management System

Students can apply for opportunities.

9.1 Submit Application

`POST /api/applications`

The backend stores:

- Student statement
- Recommendation professor
- Graduation year

The system prevents duplicate applications using:

`UNIQUE(user_id, opportunity_id)`

9.2 Retrieve Student Applications

`GET /api/applications/student/:userId`

Returns all applications submitted by a student.

9.3 Retrieve All Applications

`GET /api/applications`

Admin/manager route used for monitoring applications.

10. API Design

The backend uses RESTful API design principles.

10.1 REST Principles Used

- Stateless communication
- Resource-based URLs
- JSON request/response format
- HTTP methods for operations

10.2 HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Remove data

11. Middleware Usage

The backend uses several middleware functions.

11.1 `express.json()`

Purpose:

Parses incoming JSON request bodies.

11.2 `cors()`

Purpose:

Allows frontend and backend communication.

11.3 `express.static()`

Purpose:

Serves frontend static files.

12. Role-Based Authorization

The backend contains a custom middleware:

```
requireManager()
```

Purpose:

Restricts access to manager/admin-only operations.

The middleware checks:

```
x-user-role
```

Header value.

Allowed roles:

- admin
- manager

13. Error Handling

The backend includes structured error handling.

Examples:

Status Code	Meaning
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
409	Conflict
500	Internal server error

Example response:

```
{  
  "message": "Invalid email or password."  
}
```

14. Security Features

14.1 Password Hashing

Passwords are encrypted using bcrypt.

14.2 Email Validation

Only AIU email addresses are accepted.

Example:

@aiu.edu.eg

14.3 Role Restrictions

Administrative actions are protected.

14.4 Foreign Key Constraints

Database relationships maintain integrity.

15. Sample Backend Workflow

Student Registration Workflow

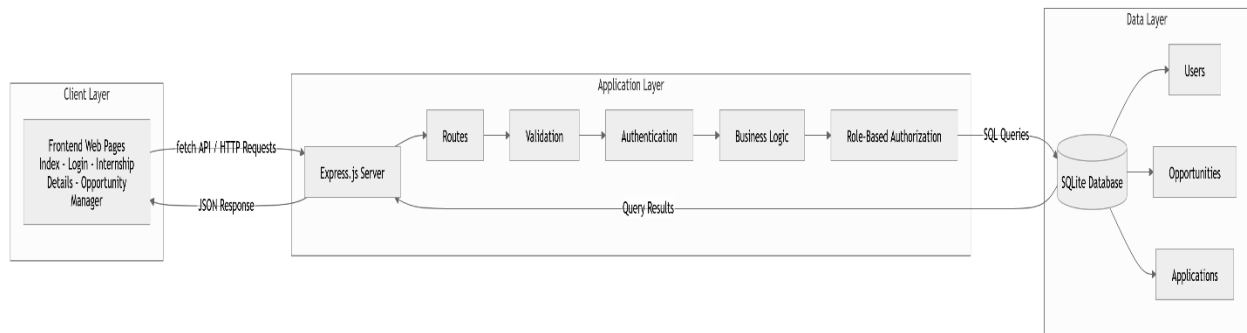
1. Student opens registration page.
2. Frontend sends POST request.
3. Backend validates data.
4. Password is hashed.
5. User stored in database.
6. Success response returned.

Opportunity Application Workflow

1. Student opens opportunity details.
2. Student submits application.

3. Frontend sends application data.
4. Backend validates request.
5. Application stored in database.
6. Confirmation response returned.

16. System Architecture:



17. Advantages of the Backend Design

Simplicity

The backend is easy to understand and maintain.

Lightweight Database

SQLite is suitable for university-scale projects.

Secure Password Storage

bcrypt improves authentication security.

Modular Structure

Separate files improve maintainability.

REST API Integration

Frontend and backend communicate efficiently.

18. Limitations of the Current System

1. No JWT authentication.
2. No session management.
3. No file upload support.
4. No email verification.
5. SQLite is less scalable for enterprise systems.
6. No rate limiting.
7. Authorization is header-based and not token-based.

19. Future Improvements

Recommended future enhancements:

Authentication Improvements

- JWT authentication
- Refresh tokens
- Multi-factor authentication

Database Improvements

- MySQL or PostgreSQL migration
- Cloud database hosting

Performance Improvements

- Caching
- Query optimization
- Pagination

Security Improvements

- HTTPS
- Rate limiting
- CSRF protection

- Input sanitization

Feature Improvements

- Notifications
- Email services
- File uploads
- Search filters
- Analytics dashboard

20. Conclusion:

The AIU Global Hub backend successfully provides the core functionality required for the platform. The system supports authentication, opportunity management, and student applications through a structured REST API architecture.

The backend demonstrates:

- Proper database design
- Secure password handling
- RESTful API implementation
- Role-based authorization
- CRUD operations
- Modular backend structure

Although the system is designed as a lightweight university project backend, it establishes a strong foundation that can later be expanded into a larger production-level system.

The project successfully integrates Node.js, Express.js, SQLite, and frontend communication to create a complete web application backend solution.

21. References:

1. Node.js Documentation
2. Express.js Documentation
3. SQLite Documentation
4. bcrypt Documentation
5. REST API Design Principles
6. MDN Web Docs